easier AI ?

timm@ieee.org

ssbse'26

july 6, 2026

code: ***pip install ezr***

credits

- **Kishan Ganguly** · *BINGO* (FSE) + *budget-aware reasoning* (arXiv).
- **Amirali Rayegan** · *simpler explanation* (JSS) + *explanation stability* (arXiv).
- **Srinath Srinivasan** · *EZR.py* (arXiv) + *neurosymbolic systems* (arXiv).

why easier AI?

how to do easier AI?

why not more of easier AI?

why easier AI?

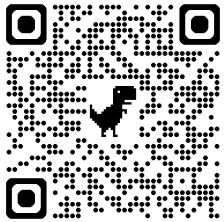
how to do easier AI?

why not more of easier AI?

Lehman’s 2nd law of software evolution

- “As a system evolves, its complexity **increases** unless work is done to maintain or reduce it.”
- Q: why / how can we **decrease** the complexity of AI?
- If anyone can do it for $1:
  - Can you do it for a cent?
- Modern debloat menu:
  - distillation, quantization, small-language-models
  - You know we can go (much) smaller, still, right?

Q: why easier AI?
A: economics

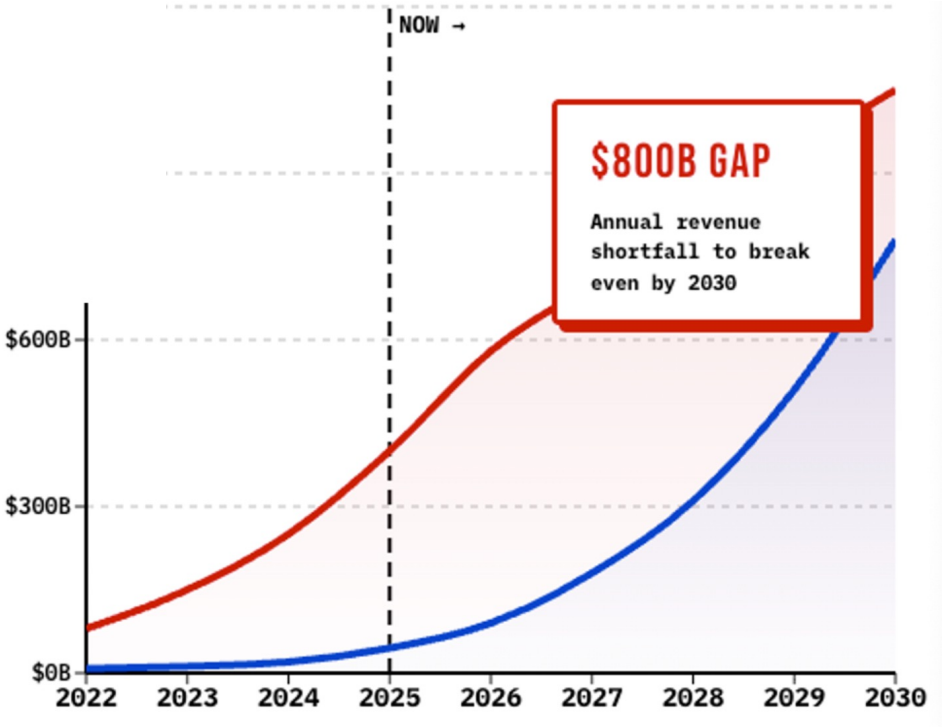


- Niklaus Wirth:
  - “Software gets **slower** faster than hardware gets faster”

- Even if $/token *falls*,
  - tasks/day and tokens/task will increase
  - +50% (or more) LLM cost (market **correction**)
  - Historically: subsidize (for market penetration), then cost: +100% (Uber)
  - *10, *100 more token burn (2023 ➔ 2026)

- Very very soon, we will be asking
  - “how **to mix and match** simpler and complex AI?”

Q: why easier AI ?
A: cause we can’t afford the alternative



- Since this 2025 forecast:
  - 2026 capex ~$725B,
  - 2027 projected >$1T
    - I.e. gap widening

Q: why easier AI?
A: trust means “we watch and understand”

My Tesla Was Driving Itself Perfectly —Until It Crashed

The danger of almost-perfect tech

By Raffi Krikorian

THE ATLANTIC
MARCH 17, 2026

Q: why easier AI?
A: cause some domains demand it

- Green AI
- global **south** (countries without e.g. American-scale infrastructure)
- resource-constraint teaching
  - community colleges,
  - K–12 CS,
  - MSIs,
  - individual using **personal** hardware
- privacy-constrained industrial development
  - **defense**,
  - regulated healthcare,
  - financial services,
  - law firms)
- air-gapped settings;
- **edge** or on-device inference

Q: why easier AI?
A: software engineering

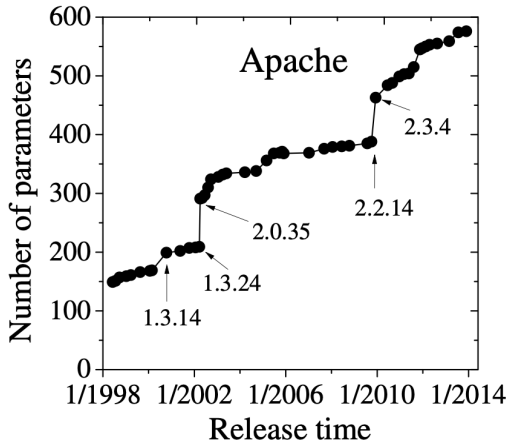
- **security**:
  - modern software = assembly of pre-made parts (> 80%)
  - more reuse == more exposure
  - e.g. left-pad; MOVEit ($12B),
    Falcon ($5B), ByBit (1.4B) $24B (2025);

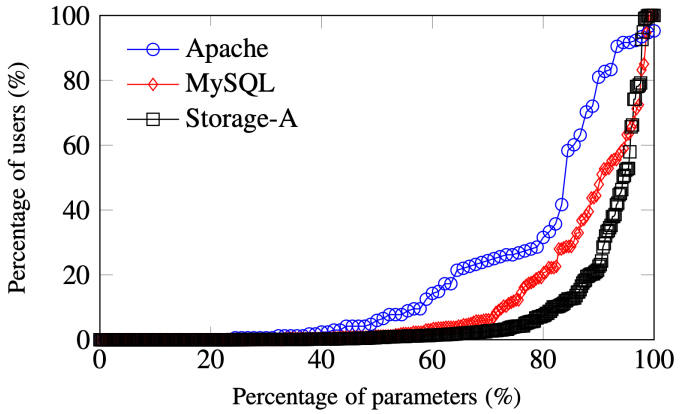
- **survivability**: e.g. SQLite does not use Berkeley DB.
  - Now used on Mars

- **maintainability**: more loc = more to maintain

- **reliability**: more loc = more that can go wrong

- **waste**: Most users cannot handle most options





Q: why easier AI?
A: cause “big AI” needs too much data

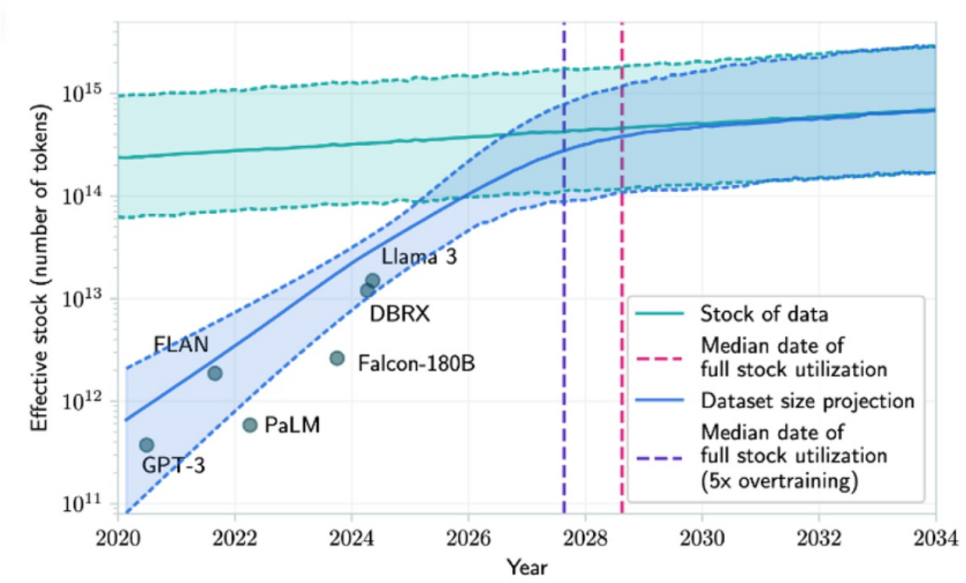
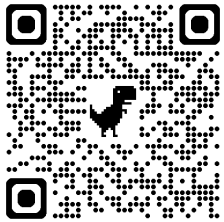


Fig. 4. Median results from the Villalobos et al. model (shown in green) estimate that by 2028, we will run out of new textual data needed to train bigger and better LLMs [75].

Q: why easier AI?
A: more AI not always better



- text mining predicting effort within **agile** projects,
  - (SVM and TFIDF) better predictions than deep learners
  - Runs 100 times faster

- text mining, programmer discussion boards
  - simpler pre-clustering the data with K-Means comparable with deep learning (500-1000 times **faster**)

- parametric statistical methods > GAN-based

- as of a 2025 lit review, many examples traditional ML methods > LLMs
  - achieve **comparable** or superior performance with significantly less computational overhead

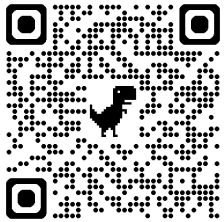
- tree-based models (Random Forest, XGBoost)
  - consistently **outperform** deep neural networks on tabular datasets,
  - especially those with more than 10,000 rows

why easier AI?

how to do easier AI?

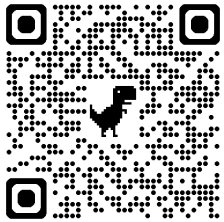
why not more of easier AI?

Q: how to do easier AI?
A: so many ways



- 130 A.D., **Ptolemy** "good principle to explain the phenomena by the simplest hypothesis possible''
- 1300: William of Ockham: "entities must not be multiplied beyond necessity"
- 1902, **PCA**: reduce data to a few principal components
- 1960s, Narrows: guide search via a few key variables
- 1974, **Prototypes**: speed up k-NN by reducing rows to a few exemplars
- 1984, JL lemma: random projection a few dimensions can preserve pairwise distances (within some error)
- 1986, **ATMS**: only focus diagnosis on core assumptions
- 1994, ISAMP: a few restarts can explore large problem spaces
- 1996, Sparse coding: learn efficient, sparse representations from data which inspired dictionary learning and **sparse autoencoders**
- 1997, Feature selection: ignore up to 80\% of features
- 2002, **Backdoors**: if we first set a few variables, that cuts exponential time to polynomial
- 2005, Semi-supervised learning: data can be approximated on a much lower-dimensional manifold
- 2009, **Active learning**: only use most informative rows
- 2003–2021, SE "keys": a few parameters govern many SE models
- 2010+, **Surrogates**: first, build small models to label the rest of the data
- 2020s, Distillation: compress large LLM models with little performance loss

Q: how to do easier AI (with active learning)
A: use current model to decide what's next



- E.g. **SMACs**
  - ensemble of decision trees
  - Seek unlabeled examples where
    - ensemble most disagrees (a.k.a. explore)
    - **ensemble** most agree (a.k.a. exploit)
    - adapt = Initially explore. Slowly shift to exploit
- Cost of SMAC3:
  - when new data arrives
  - **rebuilding** the ensemble

- EZR: **pip install ezr**
- "ensemble" = 2 **class** Bayes classifier
  - $|Best|,\ |Rest| = \sqrt{N},\quad N - \sqrt{N},$
  - **next** =
    - closest to *Best*
    - furthest from *Rest*
  - update:
    - add next to *Best*,
    - resort *Best*,
    - **pop** worst *Best* to *Rest*
  - Very fast
    - never sort *Rest*

```
Sym = dict
def Num(n=0, mu=0, m2=0): return (n, mu, m2)


def count(sym,v,inc=1):
  "Change or delete keys."
  if (c := sym.get(v,0) + inc) > 0: sym[v] = c
  else: sym.pop(v, None)
  return sym


def welford(num, v, inc=1):
  "Fold v into a Num (inc=-1 removes); return new (n,mu,m2)."
  n, mu, m2 = num
  if (n := n + inc) < 0: return Num()
  d = v - mu; mu += inc * d / n
  return (n, mu, m2 + inc * d * (v - mu))


def add(i,v,inc=1):
  "Add one value/row to i (inc=-1 removes)."
  if isinstance(i,o):
    for at,col in i.cols.items(): i.cols[at] = add(col,v[at],inc)
    (i.rows.append if inc==1 else i.rows.remove)(v)
    return i
  if v=="?": return i
  return (count if is_sym(i) else welford)(i, v, inc=inc)
```

Q: how to test EZR?
A: read lots of papers, collect their data

- Recent SBSE **papers**: ICSE, FSE, TSE, IST, EMSE, TOSEM, ASE
- 127 data sets
- 3-1044 independent **x attributes**;
- 1 to 8 dependent **y attributes** to be minimized or maximized;
- 93 to 100,000 rows.
- http://**tiny.cc/moot**

#	Category	Focus	File Name(s)	Here, optimizers struggle with issues like...	#y	#x	Rows	Refs
Software Systems & Configuration & Tuning								
24	Soft. Config	**PLE**	SS-[A-X]	Minimize footprint/memory vs. maximizing throughput	2-3	3–88	197–86k	[1]
12	PromiseTune	**Perf.**	7z, LLVM, BDBC	Minimize execution time and energy consumption	1	6–35	864–166k	[2]
1	Cloud compute	**Tuning**	Apache_AllMeas	Balance server response vs. CPU/RAM load patterns	1	9	192	[3]
1	Cloud compute	**Tuning**	SQL_AllMeas	Minimize latency/IO vs. maximizing throughput	1	39	4,654	[4]
1	Cloud compute	**Tuning**	X264_AllMeas	Optimize encoding parameters for PSNR/SSIM quality	1	16	1,153	[1]
2	Testing	**Testing**	test120, 600	Maximize coverage while minimizing execution time	1	9	5,161	–
Project Management & Process Modeling								
35	Proj. Health	**Health**	Health-Closed	Optimize PR rates and minimize developer churn	2-3	5	10,001	[5]
3	Scrum	**Agile**	Scrum[1k-100k]	Maximize velocity within sprint constraints	3	124	1k–100k	[6]
8	Feature Mod.	**Config**	FFM, FM-*	Optimize Clause/Constraint ratio in large spaces	3	128–1k	10,001	[1]
1	nasa93dem	**Cost**	nasa93dem	Minimize effort (person-months) vs. quality	3	26	93	[5]
1	COC1000	**Risk**	coc1000	Minimize risks/schedule slip vs. analyst expertise	5	20	1,001	[7]
4	POM3	**Process**	pom3[a-d]	Balance project idle rates vs. completion costs	3	9	501–20k	[6]
4	XOMO	**Defects**	xomo_[flt,grd]	Minimize defect density vs. total project effort	4	27	10,001	[7]
Interdisciplinary & Behavioral Benchmarks								
4	Behavioral	**Behavior**	all_players, etc	Maximize user retention and predict cohort churn	1-3	26–55	82–17k	[8]
4	Financial	**Finance**	BankChurners	Minimize credit risk vs. optimal pricing models	2-5	19–77	1k–20k	[9]
3	Health	**Health**	COVID19, Life	Maximize accuracy/life vs. readmission likelihood	1-3	20–64	2k–25k	[10]
2	RL	**Control**	A2C_[Acr,Crt]	Maximize reward signals vs. steps to convergence	3-4	9–11	224–318	–
5	Sales	**Market**	Marketing, socks	Maximize ROI vs. minimizing forecasting error	1-8	20–31	247–2k	[11]
3	Misc.	**General**	auto93, Car	Optimize multivariate trade-offs (e.g. MPG vs. HP)	2-5	5–38	205–1k	[1]

configuration, performance tuning, product lines, project health, defect prediction, testing, cost estimation, cross-domain generalization, and text mining

Q: how to score EZR?
A: win = 1 - normalize(regret)

- Distance to **heaven** (aggregate n goals to one)

$$d2h(y) = \sqrt{\sum_{i=1}^{k} y_i^2} \,/\, \sqrt{k},$$

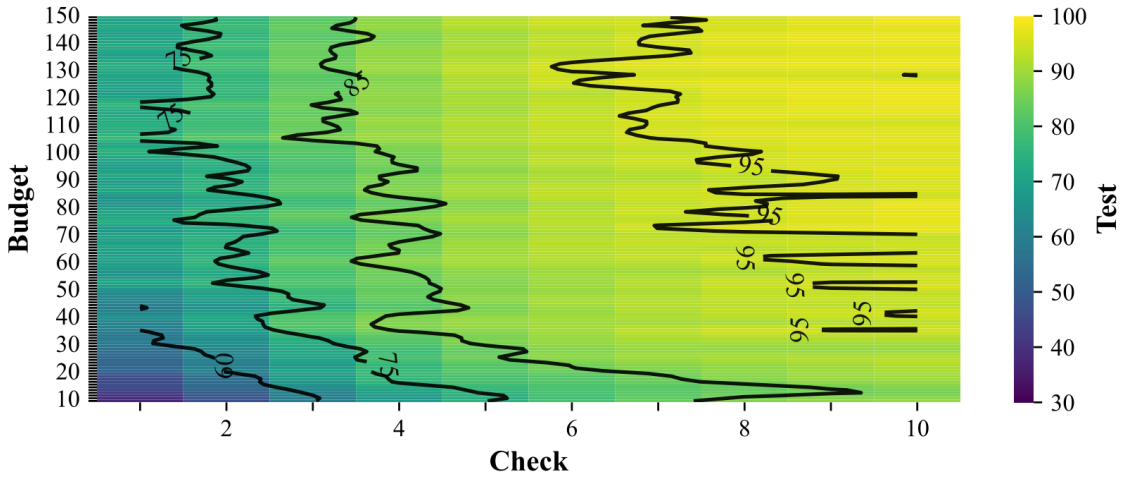
- Regret = (**d2h** – lo) / (mu – lo)
- Win = int(100 * (1 – regret))
  - Win=0 ➔ optimizer just finds mu
    - **Failure**
  - Win=100 ➔ found reference optimum
    - Success
- Build regression tree on **labeled** egs
  - Splitting rows to minimize σ of win
  - Reports what takes you towards good or bad

win	n	Lbs-	Acc+	Mpg+	
8	43	2334.2	16.3	27.4	
41	26	2038.7	17.1	30.0	Volume <= 111
71	10	1871.7	18.1	32.0	| Volume <= 83
✅ 85	4	1831.7	19.0	32.5	| | Model <= 73
62	6	1898.3	17.4	31.7	| | Model > 73
22	16	2143.0	16.5	28.7	| Volume > 83
36	7	2098.1	18.0	28.6	| | Model <= 73
12	9	2177.9	15.3	28.9	| | Model > 73
26	4	2210.0	15.7	30.0	| | | Volume > 97
0	5	2152.2	15.0	28.0	| | | Volume <= 97
-41	17	2786.1	15.1	23.5	Volume > 111
-9	5	2279.8	14.6	28.0	| Volume <= 116
-54	12	2997.1	15.3	21.7	| Volume > 116
-44	8	2921.1	15.8	22.5	| | Model > 73
-38	4	2563.0	14.0	25.0	| | | Clndrs <= 4
-50	4	3279.2	17.4	20.0	| | | Clndrs > 4
❌ -75	4	3149.0	14.4	20.0	| | Model <= 73

configuration, performance tuning, product lines, project health, defect prediction, testing, cost estimation, cross-domain generalization, and text mining

Q: how well does EZR generalize?
A: sort holdout via tree, check first few

Win = int(100 * (1 – regret))



- **100,000** times
  - pick any MOOT data set
  - Train:
    - Active learning, **labeling** y rows
    - Build tree from labels
  - Test:
    - Use tree to sort holdout
    - **Check** first few

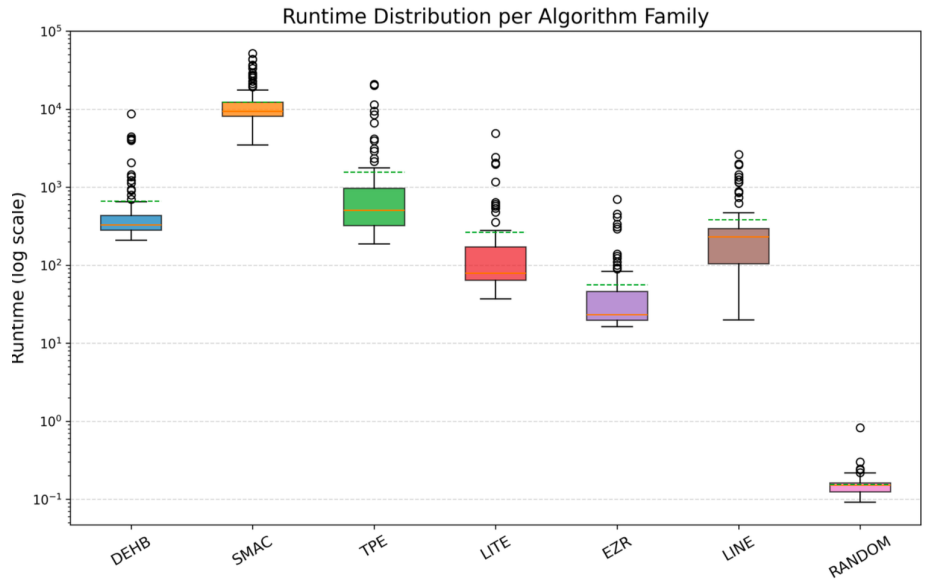
- Active learning, very effective
  - Label 50 rows
  - Check 5
  - **Achieve** 85% of best

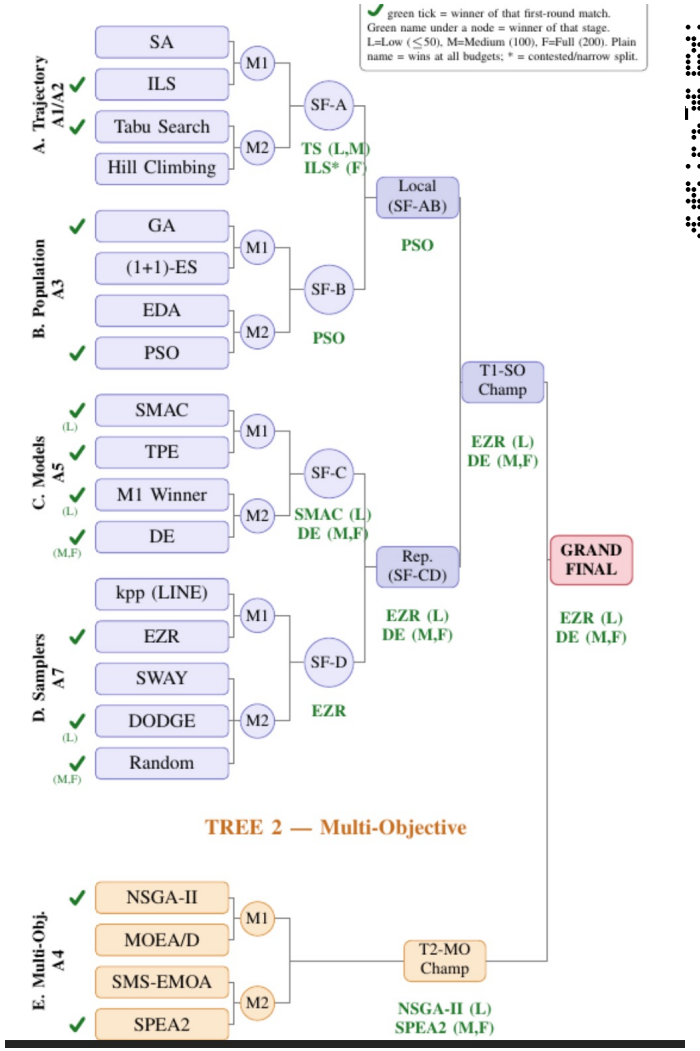
This entire 100,000-experiment study: one laptop, 2 hours, no cluster, no cloud bill.

Q: how fast and effective are EZR's trees?

A: very and very

ms

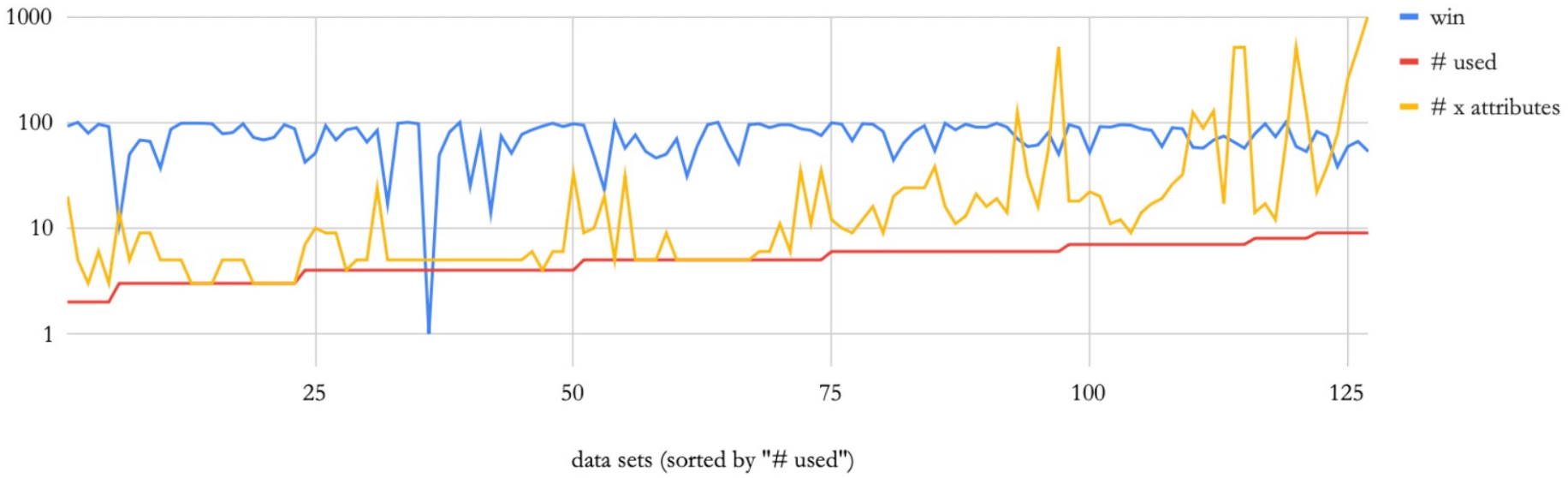




500x less CPU = 500x less energy. every run.

e.g. NSGA-II needs 1000 samples for what EZR reaches in 50

Q: how big are EZR's models
A: surprisingly small



Q: so is EZR useful for explanation?
A: EZR's trees competitive with SOTA

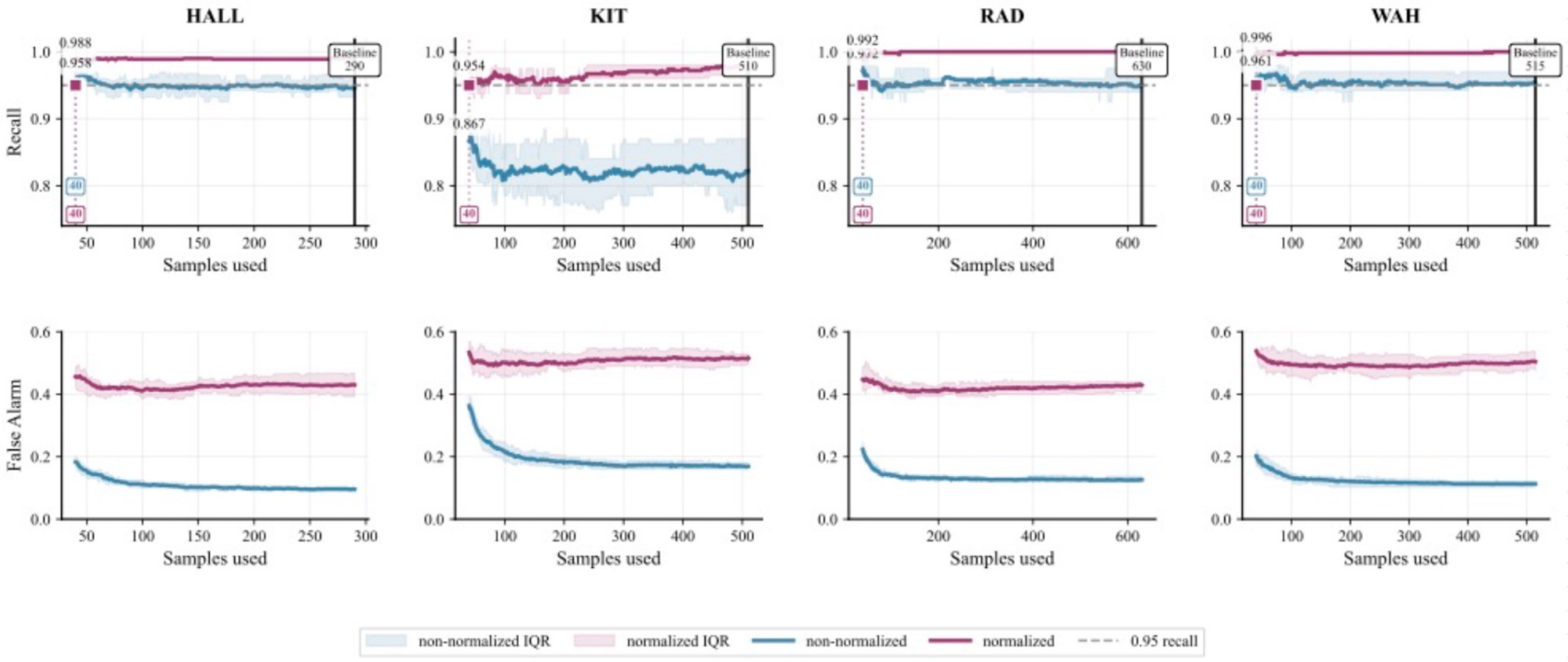
- Build trees using N **attributes**
- N= #attributes used in EZR's tree
- Using other tools:
  - select **top** N attributes

- White = tied best
- Blue = within **top** 90%
- Yellow = within top 75%
- EZR goes a long way
  - E.g. last row
  - For a **3 goal** task
    - EZR uses 7 / 1044 attributes
    - EZR uses 150 rows
    - Arguably competitive

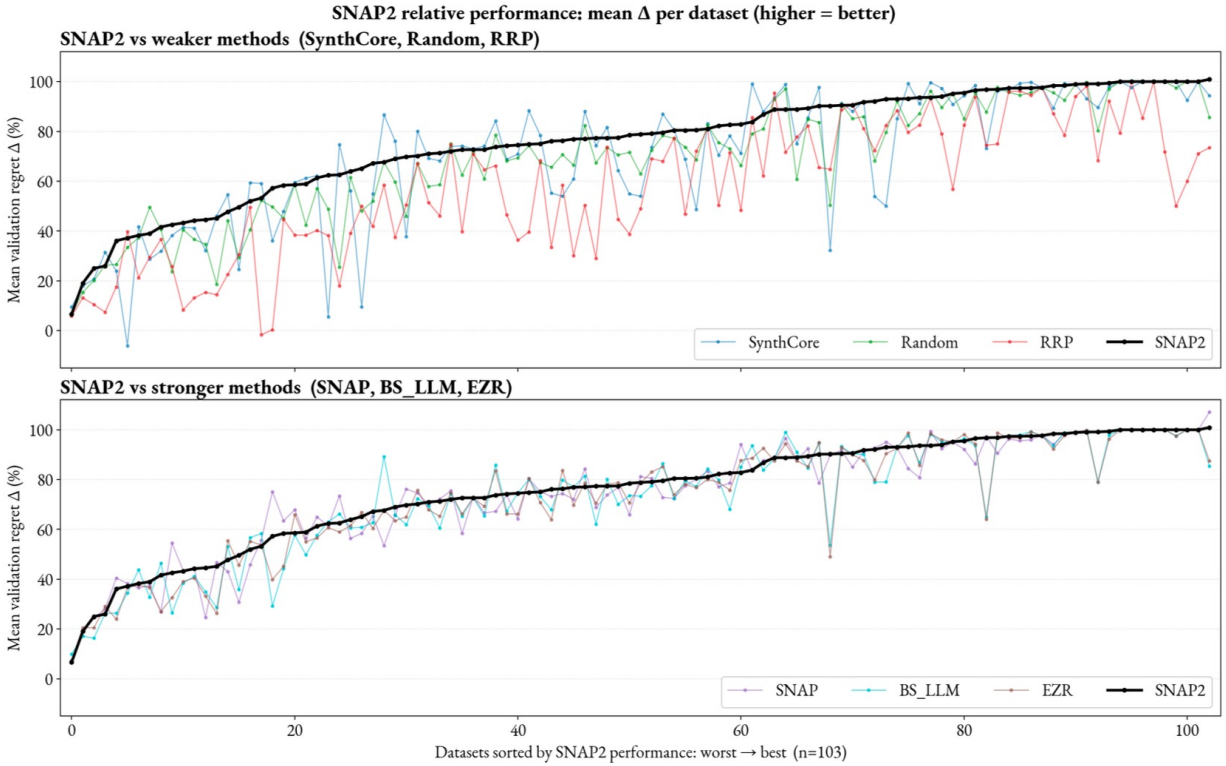
RLF	SHAP	EZR	anova	all	data	x*	x	y	budget	rows
95	100	100	100	100	HSMGP	6	14	1	150	3457
87	81	82	82	85	SS-R	7	14	2	150	3009
94	95	63	57	96	SS-V	4	16	2	150	6841
74	74	59	74	99	SS-W	7	16	2	150	65537
100	100	100	100	100	X264-AM	6	16	1	150	1153
100	100	100	99	100	SS-M	7	17	3	150	865
81	81	81	81	81	SS-N	8	17	2	150	53663
59	62	53	52	58	coc1000	9	20	5	150	1001
95	96	96	93	100	SS-U	7	21	2	150	4609
98	98	93	98	98	xomo_flight	7	27	4	150	10001
92	92	90	90	92	xomo_grnd	7	27	4	150	10001
94	95	95	95	95	xomo_osp	7	27	4	150	10001
87	88	82	82	88	xomo_osp2	8	27	4	150	10001
77	75	71	64	85	SQL-AM	10	39	1	150	4654
87	91	92	74	94	billing10k	7	88	3	150	10001
75	75	81	75	97	Scrum100k	7	124	3	150	100001
82	88	92	88	97	Scrum10k	8	124	3	150	10001
83	89	86	83	89	Scrum1k	7	124	3	150	1001
81	99	93	81	99	FFM-125	9	128	3	150	10001
81	97	89	97	97	FFM-250	8	256	3	150	10001
95	93	93	95	95	FFM-500	8	510	3	150	10001
81	96	87	96	96	FM-500-1	8	511	3	150	10001
86	91	93	91	97	FM-500-2	7	511	3	150	10001
95	95	95	87	95	FM-500-3	9	513	3	150	10001
97	97	93	96	97	FM-500-4	9	517	3	150	10001
82	94	94	71	100	FFM-1000	7	1044	3	150	10001

Q: what does EZR say about LLMs?
A: LLMs rarely tested against simpler tool

- Search google scholar for **relevant** research



Q: can EZR help LLMs?
A: yes. Hybrid LLM. EZR inits LLM's search



Method	Regime	Tops	Top-tier rate (tops/105)
SNAP2	hybrid	89	85%
SNAP	opt1	79	75%
BS_LLM	opt2	78	74%
EZR	opt3	75	71%
SYNTHCORE	opt2	73	70%
RANDOM	opt3	51	49%
RRP	opt3	34	32%

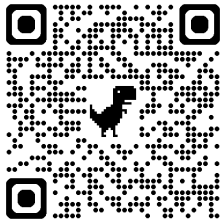
Method	Type	LLM calls	Tokens	Cost ($)	Wall (s)
RANDOM	opt3	0	0	0	0.0001
EZR	opt3	0	0	0	0.037
RRP	opt3	0	0	0	0.227
BS_LLM	opt2	1	1,800	0.0005	29.5
SYNTHCORE	opt2	1	2,000	0.0005	28.2
SNAP2	hybrid	5	19,000	0.0039	110.9
SNAP	opt1	8	26,000	0.0076	157.7

why easier AI?

how to do easier AI?

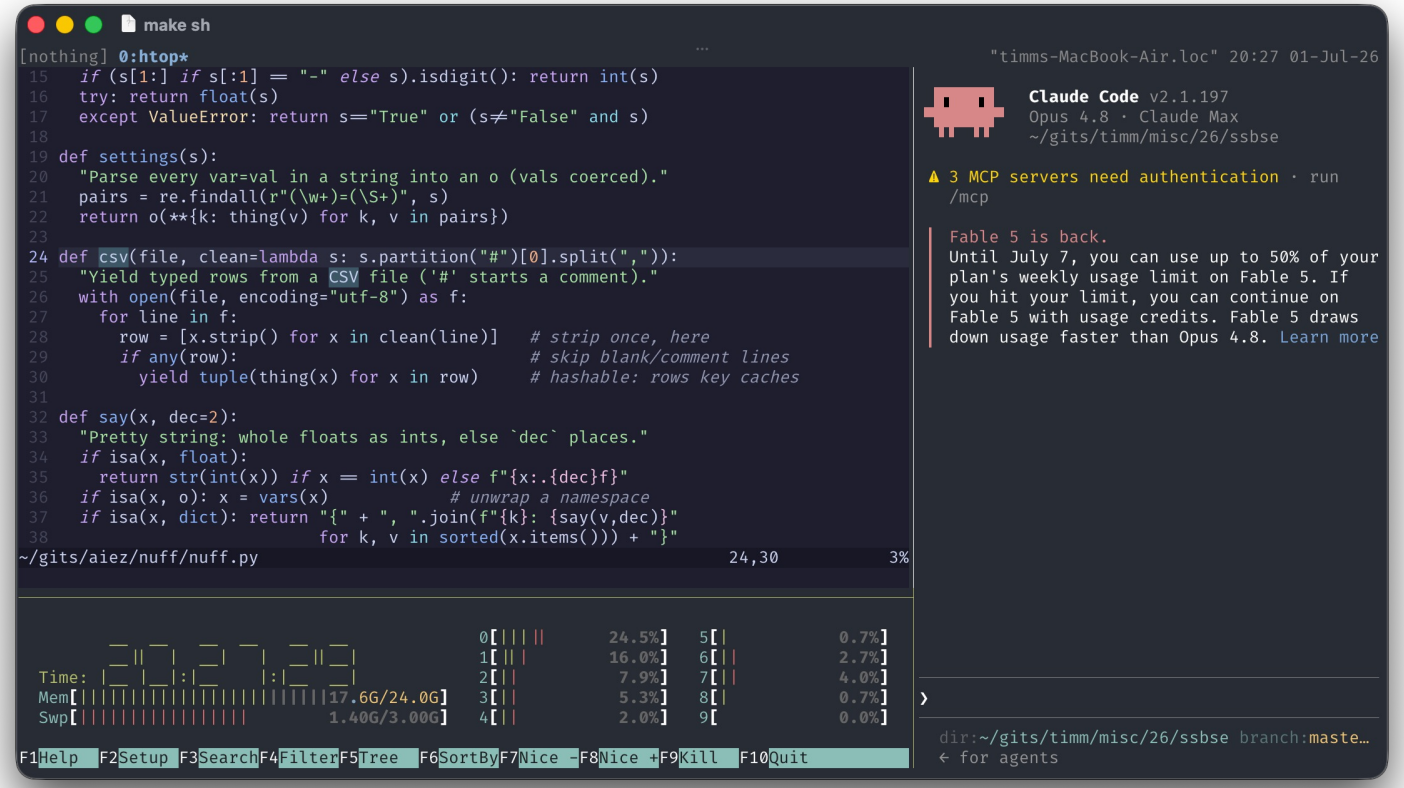
why not more of easier AI?

Q: if so good, why not more easier AI?
A: (weak) empirical standards



- In a survey of 229 SE papers on **LLMs in SE**
  - Only 5% compared to simpler approaches
- **Methodological** error
  - Other methods can produce results that are better and/or **faster**
    - (See above)

Q: if so good, why not more easier AI?
A: cause prompts/ loops/ LLM-wikis are cool

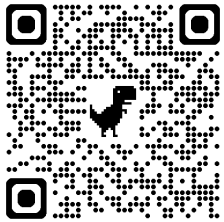


Q: if so good, why not more easier AI?
A: cause “big” sells

- Welcome to **timm inc**.
  - No special software to sell
    - cause you can build it yourself
  - No special hardware to sell
    - cause our algorithms are so fast, they don’t **need** it
  - How many shares you want?

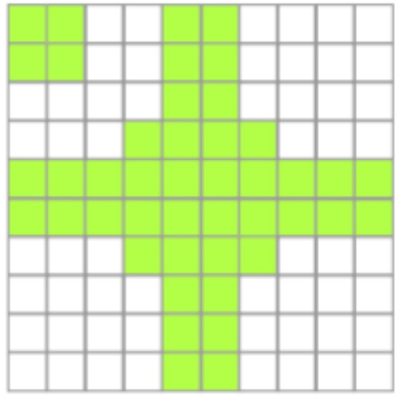
- Welcome to my data center
  - ribbon on the door the mayor can cut
  - now **that’s** a spectacle!

Q: if so good, why not easier AI?
A: the simplicity paradox: simplicity is hard



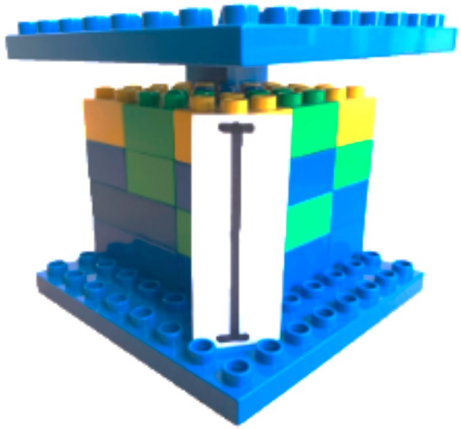
- Simplicity **research** needs a lot of CPU
  - To certify that simple(X) > complex(Y)
  - You have to build and/or **run** X and Y

Q: if so good, why not more easier AI?
A: human cognitive bias



- Our simpler methods have not been previously explored
  - since other researchers were not looking for them.
- Why?

- Humans are biased to bloat
  - In **studies** of 100s of human subjects
    - Humans favor additive to subtractive changes
    - 1,155 to 297 (about 4:1)



Conclusion

Lehman's **2nd** law of software evolution

- "as a system evolves, its complexity **increases** unless work is done to maintain or reduce it"

- Q: why / how can we **decrease** the complexity of AI?
- A: we must / we can

- **do**: baseline the simple:
  pip install ezr + tiny.cc/moot
- **review**:
  - ask "compared to what simpler?"
- **teach**:
  - easier AI first; scale only when simple fails

/ (question | comment)* /